

R.M.K GROUP OF ENGINEERING INSTITUTIONS



R.M.K
GROUP OF
INSTITUTIONS

R.M.K GROUP OF INSTITUTIONS



R.M.K
GROUP OF
INSTITUTIONS



Please read this disclaimer before proceeding:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.

22IT910
REST APPLICATION DEVELOPMENT
USING SPRING BOOT AND JPA
(Lab Integrated)

Batch / Year / Semester: 2023-2027 / III / 5

Name of the Faculty Members

RMKEC

1. Ms. V. Haritha, AP, CSE, RMKEC
2. Ms. T. Akila, AP, IT, RMKEC
3. Dr. S.D. Lalitha, ASP, CSE, RMKEC

RMDEC

1. Dr. R. Jothilakshmi , ASP, IT, RMDEC
2. Mr. P. Jagadeesan , AP, CSE, RMDEC
3. Ms. B. Pushpalatha, AP, CSE, RMDEC

RMKCET

1. Ms. P. Jemi Gold , AP, CSE, RMKCET
2. Ms. A. Sangeetha, AP, CSE, RMKCET
3. Ms. J. Jeya Rathinam , AP, CSE, RMKCET
4. Ms. J. Monisha, AP, CSE CS, RMKCET

1. CONTENTS

S.NO.	CONTENTS	PAGE NO
1	Contents	5
2	Course Objectives	6
3	Pre Requisites	7
4	Syllabus	8
5	Course Outcomes	11
6	CO- PO/PSO Mapping	12
7	Lecture Plan	13
8	Activity Based Learning	15
9	Lecture Notes	16-45
10	Assignment – 1 to 5	46-48
11	Part A Questions & Answers	49-50
12	Part B Questions	51-52
13	GATE Type Questions	53
14	Supportive Online Certification Courses	55
15	Real time Applications	56
16	Contents Beyond the Syllabus	57
17	Assessment Schedule	60
18	Prescribed Text Books & Reference Books	61
19	Mini Project – 1 to 5	62

2. COURSE OBJECTIVES

OBJECTIVES:

The Course will enable learners:

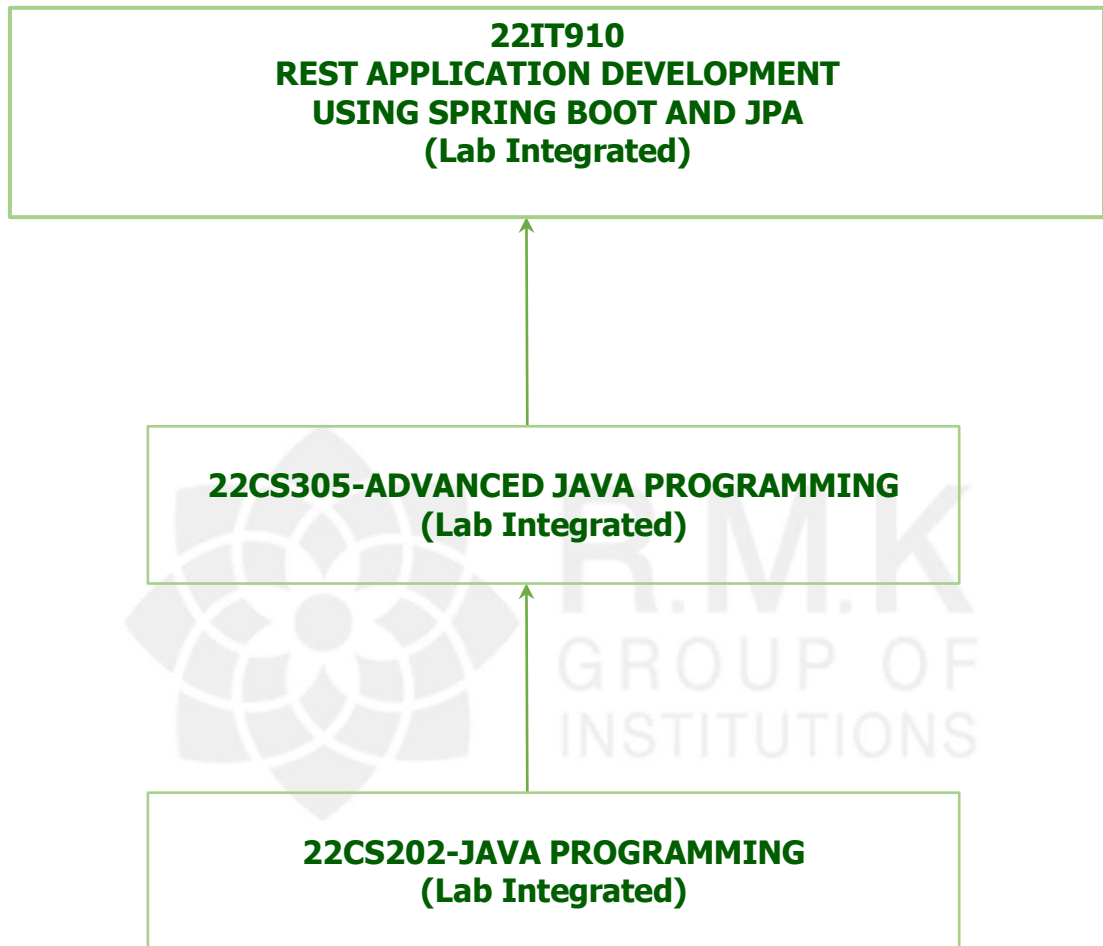
OBJECTIVES:

The Course will enable learners to:

- Provide comprehensive knowledge of RESTful APIs and the HTTP methods used in the Spring Boot framework.
- Cover advanced querying techniques using JPA, including LIKE queries, and to manage CRUD operations using JPQL.
- Explore various relational mappings in JPA, such as one-to-one and one-to-many associations, and their practical implementations.
- Implement and manage Spring AOP applications using annotation-based configurations for method interception and post-execution operations.
- Build production-grade Spring Boot applications with integrated security using JWT, detailed API documentation with SwaggerUI and OpenUI, and effective logging practices.

3. PRE REQUISITES

- **Pre-requisite Chart**



4. Syllabus

REST APPLICATION DEVELOPMENT USING SPRING BOOT AND JPA (Lab Integrated) – PEC – 4 - L T P C - 2 0 2 3

UNIT I - INTRODUCTION TO REST API 6+6 hours

RESTful APIs – overview about data exchange between client and server - separating concerns between handling HTTP requests and executing business logic - retrieving server resources via HTTP requests - injection of property values - self-contained application - serialization and deserialization – JSON properties - managing data access.

List of Exercise/Experiments

1. Develop a RESTful API for retrieving a welcome message, emphasizing the basics of data exchange between client and server.
2. Implement a RESTful API to acknowledge the user's favorite color choice, highlighting property value injection principles.
3. Create a Spring Boot application that retrieves and displays application information, demonstrating the usage of the @Value annotation to inject property values from the application configuration file.
4. Construct a RESTful API for student details retrieval, illustrating the utilization of @JsonIgnore annotation, focusing on advanced JSON property handling and data access control

UNIT II - ADVANCED DATA MANAGEMENT WITH JAVA AND MYSQL 6+6 hours

Build production-grade applications – MYSQL - mapping Java classes to relational database - repository interface - data access operations – retrieving data from the database – mapping of request body to entity - retrieve an entity - capture data from API requests - building complex queries using keywords.

List of Exercise/Experiments

1. Develop a web application for managing patient details using RESTful APIs, implementing POST and GET operations.
2. Create a web application for managing product details using RESTful APIs, enabling POST and GET operations.
3. Build an application for managing employee details using RESTful APIs, supporting POST, PUT, and DELETE operations.

4. Syllabus

UNIT III - ADVANCED JPA QUERIES AND ANNOTATIONS 6+6 hours

Pagination & Sorting using JPA, @Transient Annotation, Queries using JPA, Starts and Ends with query using JPA, JPQL with @Query Annotation, custom JPQL queries.

List of Exercise/Experiments

1. Develop a web application for pagination and sorting of children details using RESTful APIs, implementing POST and GET operations.
2. Create a web application for managing Person details using JPA methods via RESTful APIs, enabling POST and GET operations.
3. Retrieve person details using JPQL with conditions for names starting or ending with specific patterns.
4. Build a web application for managing Person details using custom JPQL queries via RESTful APIs, supporting POST and GET operations.

UNIT IV JPA ASSOCIATIONS AND MAPPING 6+6 hours

JPA Mapping of One-to-One Associations - fetching entities using queries – Loading optimization technique - Two-way One-to-One Relationship Mapping with JPA - single entity instance associated with multiple instances - Adding Data with One-to-One and One-to-Many Associations using JPA.

List of Exercises/Experiments:

1. Develop a Spring Boot application with "Person" and "Address" entities, where each person has exactly one address. Utilize Spring JPA to establish a one-to-one mapping between these entities.
2. Create a Spring Boot application with "Author" and "Book" entities, where each author can have multiple books, and each book belongs to only one author. Use Spring JPA to establish a one-to-many bidirectional mapping between these entities.
3. Build a Spring Boot application with "Employee" and "Address" entities, ensuring that each employee has exactly one address, and each address belongs to only one employee. Establish a one-to-one mapping between these entities using Spring JPA and utilize the Criteria API to retrieve employee details efficiently.

4. Syllabus

UNIT V SPRING BOOT ESSENTIALS: API SECURITY, LOGGING, AOP, AND BUILD MANAGEMENT 6+6 hours

SwaggerUI with Spring Boot, OpenUI with Spring Boot, Logging with Spring Boot, Changing Log Level, Logging Request and Response- Managing Spring Boot Logging Configuration - AspectOriented Programming (AOP) Concepts – Method Parameter Handling - Post- Execution Operations - Returning Data Handling - Comprehensive Advice Handling. API security using JWT, Gradle for build management, Sonar Lint for coding standards and guidelines.

List of Exercises/Experiments:

1. Develop a web application for managing Employee and Payroll details via RESTful APIs. Utilize Spring JPA to establish a one-to-one mapping between Employee and Payroll entities. Demonstrate the usage of Swagger for API documentation and interaction.
2. Develop a Spring Boot application focused on handling person details and integrate comprehensive logging capabilities to track application activities effectively.
3. Explore the implementation of Aspect-Oriented Programming (AOP) in a Spring application to enhance the behavior of a service method and demonstrate its impact on application functionality.

TOTAL: 30+30=60 PERIODS

LIST OF EQUIPMENTS/SOFTWARE:

Java Persistence API, Spring Boot

5. Course Outcomes

Course Outcomes with Knowledge Levels:

CO1: Create simple applications using RESTful APIs and effectively manage HTTP methods within the Spring Boot framework. – K3

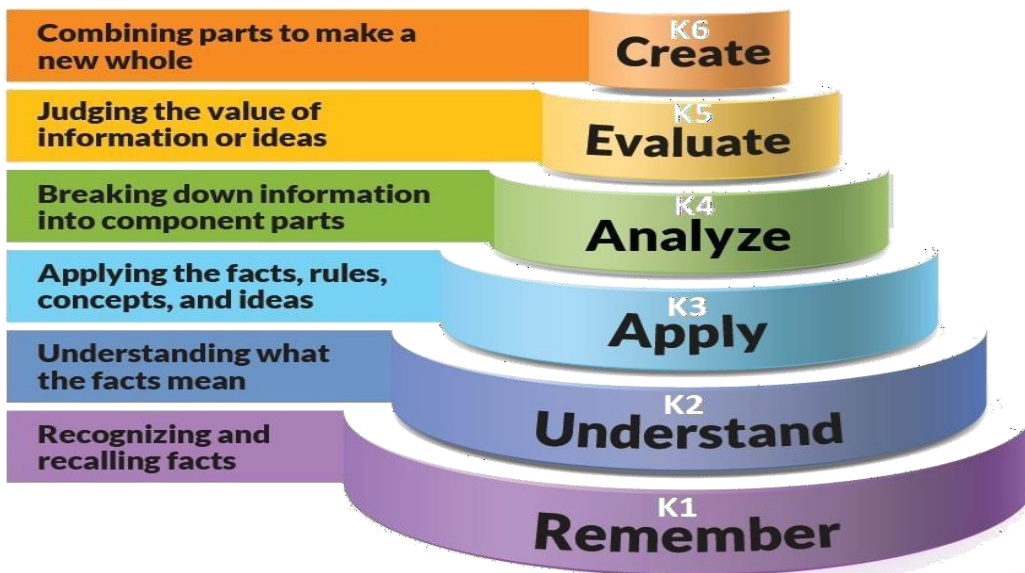
CO2: Apply database connectivity with JPA, utilizing advanced queries to interact with the database. – K4

CO3: Build applications using Spring Boot and perform CRUD operations efficiently using JPQL. – K4

CO4: Demonstrate the implementation of various relational mappings in JPA, including one- to one and one-to-many associations. – K4

CO5: Develop real-time applications that integrate user interfaces and utilize Spring AOP for method interception and advice handling. – K5

CO6: Apply security measures to REST APIs using Spring Security and JWT to protect sensitive data and ensure secure communication between clients and servers. – K5



6. CO-PO/PSO Mapping

CO	PROGRAM OUTCOMES												PSO		
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
C01	3	2	3	-	2	-	-	-	-	1	-	2	2	3	2
C02	3	3	3	-	2	-	-	-	-	-	-	2	3	3	2
C03	3	3	3	-	2	-	-	-	-	1	-	2	3	3	2
C04	3	2	3	-	2	-	-	-	-	-	-	2	2	3	2
C05	3	2	3	-	3	-	-	2	1	2	-	3	2	3	3
C06	3	2	3	-	2	-	-	3	2	2	-	3	2	3	2
CO	3.00	2.33	3.00	-	2.17	-	-	2.50	1.50	1.50	-	2.33	2.33	3.00	2.17

Correlation Level

1. Slight (Low)
2. Moderate (Medium)
3. Substantial (High) ,

If there is no correlation, put "-".

7. LECTURE PLAN

Sl. No.	Topics	No. of Periods	Proposed Date	Actual Date	Pertaining CO	Taxonomy Level	Mode of Delivery
1	Pagination & Sorting using JPA	1	As Per Academic Calendar	As Per Academic Calendar	CO3	K4	MD 1, MD 4
2	@Transient Annotation	1	As Per Academic Calendar	As Per Academic Calendar	CO3	K4	MD 1, MD 4
3	Queries using JPA	1	As Per Academic Calendar	As Per Academic Calendar	CO3	K4	MD 1, MD 4
4	Starts and Ends with query using JPA	1	As Per Academic Calendar	As Per Academic Calendar	CO3	K4	MD 1, MD 4
5	JPQL with @Query Annotation	1	As Per Academic Calendar	As Per Academic Calendar	CO3	K4	MD 1, MD 4
6	custom JPQL queries	1	As Per Academic Calendar	As Per Academic Calendar	CO3	K4	MD 1, MD 4
7-	Lab Exercises - Develop a web application for pagination and sorting of children details using RESTful APIs, implementing POST and GET operations. - Create a web application for managing Person details using JPA methods via RESTful APIs, enabling POST and GET operations.	6	As Per Academic Calendar	As Per Academic Calendar	CO3	K4	iamneo



	3. Retrieve person details using JPQL with conditions for names starting or ending with specific patterns. 4. Build a web application for managing Person details using custom JPQL queries via RESTful APIs, supporting POST and GET operations. iamneo exercises						
--	---	--	--	--	--	--	--

* MD1 – Oral Presentation , * MD 4 – Hands on using any IDE

8. Activity Based Learning

Common Activities

Learning Methods	Activities
Learn by Solving Problems	Class Exercises, Challenge Yourself, Practice At Home exercises posted in iamneo Portal
Learn by Questioning	Knowledge Check / MCQ Using iamneo portal and RMK Nextgen App
Learn by Hands on	Practice available in iamneo Portal

iamneo

RMK Nextgen[®]

Unit III Activity : - "Build a Product Catalog API"

Objective:

Develop a simple **Spring Boot + JPA** REST API for managing a product catalogue that demonstrates pagination, sorting, custom queries, and use of @Transient.

Activity Description:

1. Entity Design – Product
Create a Product entity with required fields
2. Repository – ProductRepository
Extend JpaRepository<Product, Long>
Add custom JPQL query methods:
3. Pagination & Sorting
Implement an endpoint
4. Controller – ProductController
Create REST endpoints:

9. Lecture Notes

Unit III

Java Persistence API

9. Lecture Notes

Pagination & Sorting using JPA, @Transient Annotation, Queries using JPA, Starts and Ends with query using JPA, JPQL with @Query Annotation, custom JPQL queries.

PAGINATION & SORTING USING JPA

In Java Persistence API (JPA), pagination and sorting are crucial features when dealing with large datasets retrieved from a database. Pagination allows fetching a subset of data, typically to enhance performance by reducing the amount of data transferred and displayed at once.

Sorting, on the other hand, enables ordering query results based on specified criteria, such as ascending or descending order of columns.

Pagination

Pagination involves dividing query results into manageable chunks, or pages, to improve performance and user experience. By specifying a starting point (setFirstResult) and maximum number of results (setMaxResults), developers can control which subset of data to retrieve and display.

Sorting

Sorting arranges query results in a specified order, such as ascending or descending, based on one or more attributes of the entities. This can be achieved using the ORDER BY clause in JPQL (Java Persistence Query Language) queries or through the CriteriaQuery API in JPA.

Reasons Why We Are Using Pagination And Sorting:

We should use the pagination and sorting techniques for the following reasons:

1. Small results are easier to process by both server and the client.
2. Large results would require more resources(CPU,MEMORY,NETWORK etc)
3. Sorted results bring order to the system.
4. Sorting makes it possible to prioritize the items.

In a Spring Boot application, implementing pagination and sorting in a REST API can greatly improve the efficiency and usability of the API, especially when dealing with large datasets. Spring Data JPA provides built-in support for pagination and sorting through the `Pageable` and `Sort` interfaces.

Here's a step-by-step guide on how to implement pagination and sorting in a Spring Boot REST API.

1. Setup Your Spring Boot Application

Ensure you have the necessary dependencies in your `pom.xml` file:

```
```.xml
<dependencies>
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
 <groupId>com.h2database</groupId>
 <artifactId>h2</artifactId>
 <scope>runtime</scope>
</dependency>
</dependencies>
```.
```

2. Create an Entity

Define an entity class that maps to a database table. For this example, we'll use a simple `Product`

```
```java
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;

@Entity
public class Product {

 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private Long id;
 private String name;
 private double price;
 // Getters and setters
}
```
```

3. Create a Repository

Create a repository interface that extends `JpaRepository` to gain access to pagination and sorting

```
```java
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface ProductRepository extends JpaRepository<Product, Long>
{
}
```
```

4. Create a Service

Create a service class to handle the business logic:

```
```java
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;

@Service
public class ProductService {

 @Autowired
 private ProductRepository productRepository;

 public Page<Product> getProducts(Pageable pageable) {
 return productRepository.findAll(pageable);
 }
}
```
```

5. Create a Controller

Create a controller to handle incoming HTTP requests and use the service to fetch paginated and sorted data.

```
```java
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/products")
public class ProductController {
```

```

public class ProductController {

 @Autowired
 private ProductService productService;

 @GetMapping
 public Page<Product> getProducts(@PageableDefault(size = 10, sort = "name") Pageable
 pageable) {

 return productService.getProducts(pageable);

 }

}

```

## 6. Configuration

Optionally, you can configure some default pagination settings in your `application.properties` file:

```

` `` `properties
spring.data.web.pageable.default-page-size=10
spring.data.web.pageable.max-page-size=50
` `` `

```

## Example Requests

You can test the API using different query parameters for pagination and sorting:

- **Default Request:** `GET /products`
- **Custom Page Size:** `GET /products?page=0&size=5`
- **Sorting by Price:** `GET /products?sort=price,asc`
- **Pagination and Sorting:** `GET /products?page=1&size=5&sort=price,desc`

With these steps, you have implemented pagination and sorting in your Spring Boot REST API. The `Pageable` interface makes it easy to handle pagination, and the `Sort` interface allows for flexible sorting of results. This setup ensures your API can handle large datasets efficiently and provides a better experience for clients consuming the API.

# @TRANSIENT ANNOTATION

The @Transient annotation is used in Java to indicate that a particular field should not be persisted in the database. This annotation is typically used in the context of Java Persistence API (JPA) and Hibernate, where it plays a crucial role in managing the mapping of class attributes to database columns.

When a field is marked with @Transient, the persistence provider will ignore that field during any database operations such as insert, update, or select.

## Key points

- ⌚ Non-Persisted Field
- ⌚ Serialization
- ⌚ Data Integrity

## Purpose

The primary purpose of using the @Transient annotation is to exclude specific fields from being persisted in the database. This can be useful in various scenarios, such as when a field is derived or calculated from other persisted fields, or when it holds temporary state information that should not be saved.

## Realtime Example

Consider a scenario in an e-commerce application where an entity Order has a field totalAmount which is calculated from the sum of its line items. Since totalAmount is a derived value, it should not be persisted in the database.

Therefore, it can be annotated with @Transient to exclude it from persistence operations.

## Why use @Transient

- ⌚ **Efficiency:** Reduces the amount of data stored in the database, leading to potentially faster query performance and less storage usage.
- ⌚ **Clarity:** Clearly distinguishes between fields that are part of the entity's persistent state and those that are not, improving code readability and maintainability.
- ⌚ **Control:** Gives developers fine-grained control over the mapping between the application's data model and the database schema.

The @Transient annotation is a powerful tool for managing the persistence of Java objects, ensuring that only necessary data is stored while keeping derived or temporary data transient.

Here is a simple example demonstrating the use of the `@Transient` annotation:

Entity Class with `@Transient` Field

```
```java
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Transient;

@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private double price;
    @Transient
    private double discountedPrice;
    // Getters and setters
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

```

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public double getDiscountedPrice() {
        return discountedPrice;
    }

    public void setDiscountedPrice(double discountedPrice) {
        this.discountedPrice = discountedPrice;
    }

    // Business logic to calculate discounted price
    public void applyDiscount(double discountRate) {
        this.discountedPrice = this.price - (this.price * discountRate);
    }
}
```

```

## Explanation

- **Entity Annotation** (`@Entity`): Marks the class as a JPA entity.
- **ID Annotation** (`@Id`): Marks the primary key field.
- **Generated Value Annotation** (`@GeneratedValue`): Specifies how the primary key should be generated.
- **Transient Annotation** (`@Transient`): Marks the `discountedPrice` field as transient, meaning it will not be persisted in the database.

## Testing the Entity

When you persist an instance of `Product`, the `discountedPrice` field will not be stored in the database. However, you can still use it within your application for any temporary calculations or business logic.



## Example Test

```
```java
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class DataLoader implements CommandLineRunner {

    @Autowired
    private ProductRepository productRepository;

    @Override
    public void run(String... args) throws Exception {

        Product product = new Product();
        product.setName("Example Product");
        product.setPrice(100.0);
        // Apply a discount
        product.applyDiscount(0.1); // 10% discount
        System.out.println("Discounted Price: " + product.getDiscountedPrice());
        // Save to the database
        productRepository.save(product);
        // Load from the database
```

```
Product savedProduct = productRepository.findById(product.getId()).orElse(null);
```

```
if (savedProduct != null) {
```

```
    System.out.println("Loaded Product: " + savedProduct.getName());
```

```
    System.out.println("Price: " + savedProduct.getPrice());
```

```
    // Note: The discountedPrice will be 0.0 as it is not persisted
```

```
    System.out.println("Discounted Price: " + savedProduct.getDiscountedPrice());
```

```
}
```

```
}
```

```
}
```

In this example, the `discountedPrice` will not be persisted and will be recalculated or reset as needed when the entity is used within the application.

Conclusion

The `@Transient` annotation is a useful tool in JPA for indicating fields that should not be persisted in the database. It allows you to manage data within your application without affecting the database schema or storage requirements. Use it for temporary, calculated, or non-persistent fields to keep your database clean and efficient.

Queries using JPA

Java Persistence API (JPA) provides a powerful and flexible way to perform database operations using Java objects. With JPA, you can create and execute queries to retrieve, update, or delete data from the database.

JPA supports both JPQL (Java Persistence Query Language) and native SQL queries, offering a range of options to interact with the database efficiently.

Key points

JPQL

Criteria API

Named Queries

Native Queries

Example

```
public List<Book> findBooksByAuthor(EntityManager em, String author) {  
    String jpql = "SELECT b FROM Book b WHERE b.author = :author";  
    TypedQuery<Book> query = em.createQuery(jpql, Book.class);  
    query.setParameter("author", author); return query.getResultList();  
}
```

Why we use JPA Queries

Abstraction: Simplifies database operations by allowing developers to work with entity objects instead of SQL statements.

Portability: JPQL queries are database-agnostic, making the application more portable across different database systems.

Productivity: Named queries and the Criteria API can improve development productivity and maintainability.

Queries Using JPA: An Overview

Java Persistence API (JPA) is a specification that provides a framework for managing relational data in Java applications. Queries in JPA are used to retrieve, update, or delete data from a database. Here's an overview of how queries work in JPA:

1. JPQL (Java Persistence Query Language)

What is JPQL?

- JPQL is an object-oriented query language similar to SQL but operates on the entity objects and their relationships rather than directly on database tables.
- It supports a rich set of operations including SELECT, UPDATE, DELETE, joins, aggregations, and more.

Example JPQL Query:

```
@Query("SELECT e FROM Employee e WHERE e.department = :dept")
```

```
List<Employee> findEmployeesByDepartment(@Param("dept") String department);
```

- This query selects all Employee entities where the department matches the specified value.

2. Native Queries

What are Native Queries?

- Native queries allow you to write raw SQL queries, bypassing JPQL and directly interacting with the database. This can be useful for complex queries that are difficult to express in JPQL or for performance optimization.

Example Native Query:

```
@Query(value = "SELECT * FROM employees WHERE department = ?1", nativeQuery = true)
```

```
List<Employee> findEmployeesByDepartment(String department);
```

- Here, raw SQL is used to find employees based on their department.

3. Derived Query Methods

What are Derived Query Methods?

- JPA allows you to create queries by simply defining method names in your repository interface. The method names follow a specific pattern and JPA derives the query based on the method name.

Common Patterns:

- `findBy<FieldName>`: Find records by a specific field.
- `countBy<FieldName>`: Count records matching a specific field.
- `deleteBy<FieldName>`: Delete records matching a specific field.

Examples:

```
List<Employee> findByLastName(String lastName); // Find employees by last name
```

```
List<Employee> findByFirstNameStartingWith(String prefix); // Find employees with first name starting with prefix
```

```
List<Employee> findByEmailEndingWith(String suffix); // Find employees with email ending with suffix
```

Pagination and Sorting

4. Pagination:

- JPA provides support for pagination through the Pageable interface. You can limit the number of records returned and define the starting point.

5. Sorting:

- Sorting can be handled by adding the Sort parameter to your query method or using OrderBy in JPQL.

Example with Pagination and Sorting:

```
Page<Employee> findByDepartment(String department, Pageable pageable); // Pageable provides pagination and sorting
```

@Query Annotation

Purpose:

- The @Query annotation is used to define custom queries. It can be applied to methods in repository interfaces, allowing you to write JPQL or SQL directly within your Java code.

Usage:

- Useful for complex queries that can't be derived from method names or when performance optimization is needed.

Example:

```
@Query("SELECT e FROM Employee e WHERE e.salary > :salary")
```

```
List<Employee> findHighEarningEmployees(@Param("salary") double salary);
```

6. Named Queries

What are Named Queries?

- Named queries are pre-defined, static queries that can be defined using the @NamedQuery annotation or within XML files. They are defined at the entity level and can be reused across multiple repositories.

Example:

```
@NamedQuery(  
    name = "Employee.findAll",  
    query = "SELECT e FROM Employee e"  
)
```

- Named queries are precompiled, making them efficient for repeated use.

7. @Modifying Annotation

Purpose:

- The @Modifying annotation is used to indicate that a query method is performing an update or delete operation. Without this annotation, update or delete operations in custom queries will not execute.

Example:

@Modifying

@Query("UPDATE Employee e SET e.salary = :salary WHERE e.id = :id")

int updateEmployeeSalary(@Param("salary") double salary, @Param("id") Long id);

Query Parameters

Positional Parameters:

- Parameters are passed by their position in the query using ?1, ?2, etc.

Named Parameters:

- Parameters are passed by name using :paramName.

Example:

@Query("SELECT e FROM Employee e WHERE e.firstName = :firstName AND e.lastName = :lastName")

List<Employee> findByFullName(@Param("firstName") String firstName, @Param("lastName") String lastName);

Queries in JPA are powerful and flexible, allowing developers to retrieve and manipulate data in an object-oriented way. By using JPQL, native queries, derived methods, and annotations like @Query and @Modifying, you can efficiently interact with your database while keeping your code clean and maintainable.

Starts and Ends with query using JPA

- In JPA, querying with string patterns such as "starts with" and "ends with" can be accomplished using JPQL's LIKE operator. This is particularly useful for searching text fields where the values begin or end with a specific substring.
- These queries enhance the flexibility of database searches, making it easier to filter results based on partial matches.

Key points

- LIKE Operator
- Wildcards
- Starts With
- Ends with

Real-time Example

- Imagine an application managing a customer database for an e-commerce platform. The platform needs to filter customer records based on their email addresses.
- For instance, the system can identify customers whose email addresses start with "info" or end with "example.com".

Starts with Scenario

- The platform wants to find all customers who have generic email addresses starting with "info". This can help in identifying and managing business-related email accounts.

Ends with Scenario

- To ensure compliance with a regional marketing campaign, the platform needs to retrieve all customer email addresses ending with "example.com". This helps in targeting a specific audience whose email domain matches the company's domain.

- By leveraging the LIKE operator in JPQL, the e-commerce platform can efficiently query and manage customer data based on these specific string patterns, enhancing both operational efficiency and targeted marketing efforts.

JPQL with @Query Annotation

- The @Query annotation in Java is used in conjunction with JPQL (Java Persistence Query Language) to define custom queries directly on repository methods within Spring Data JPA.
- This powerful feature allows developers to write complex queries without needing to create separate query methods, providing a clean and efficient way to interact with the database.

Key points

- Custom Queries
- Flexibility
- Parameter Binding
- Readability

Example

- Consider a scenario in a customer management system where you need to find all customers who have made purchases over a certain amount in the last year. Instead of writing multiple methods or complex query logic in the service layer, you can define a custom query using the @Query annotation in your repository.
- In this example, the @Query annotation allows you to specify a JPQL query that filters customers based on purchase history and amount, ensuring that only relevant data is fetched from the database. This approach simplifies the repository interface while providing the flexibility to handle complex query requirements.

Why use JPL with @Query

- 🕒 **Efficiency:** Simplifies the process of writing and managing complex queries.
- 🕒 **Conciseness:** Reduces boilerplate code and keeps the repository interface clean.
- 🕒 **Maintainability:** Enhances code readability and maintainability by centralizing query definitions.

Using JPQL with the @Query annotation enables efficient and flexible data retrieval, making it an essential tool for developers working with complex database interactions in Java applications.

Custom JPQL Queries

Custom JPQL (Java Persistence Query Language) queries allow developers to perform complex database operations tailored to specific application requirements. JPQL is similar to SQL but operates on the entity model rather than directly on database tables.

Custom JPQL queries enable fine-grained control over data retrieval and manipulation, enhancing the flexibility and power of JPA.

Key points

- 🕒 Entity-Centric
- 🕒 Flexibility
- 🕒 Portability
- 🕒 Reusability

Example

Consider an e-commerce application that needs to generate a sales report. The report requires filtering orders based on various criteria such as date range, customer status, and order total. Using custom JPQL queries, the application can dynamically build these queries to fetch the required data.

For instance, the sales manager wants to see all orders placed by VIP customers in the last month with a total value exceeding \$500. A custom JPQL query can be crafted to join the Order and Customer entities, apply the necessary filters, and aggregate the results.

Why use Custom JPQL Queries

- 🕒 **Precision:** Tailors database interactions precisely to application needs, ensuring optimal data retrieval.
- 🕒 **Complex Conditions:** Capable of handling intricate conditions and multiple entity relationships.
- 🕒 **Scalability:** Supports building scalable applications by allowing fine-tuned queries and efficient data access patterns.

Custom JPQL queries are essential for developing sophisticated and responsive Java applications, enabling developers to harness the full potential of JPA for complex data operations.

Starts and Ends with query using JPA

To perform "Starts with" and "Ends with" queries using JPA (Java Persistence API), you can use the LIKE keyword with the appropriate wildcards in your JPQL (Java Persistence Query Language) or native SQL queries.

Here's how you can do it:

Example Entity

Let's assume you have an entity called Person with a field name.

@Entity

```
public class Person {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String name;
```

```
    // getters and setters
```

```
}
```

1. Starts with Query

To find all Person entities where the name starts with a specific string (e.g., "John"), you can use:

```
@Query("SELECT p FROM Person p WHERE p.name LIKE :namePrefix%")
```

```
List<Person> findByNameStartsWith(@Param("namePrefix") String namePrefix);
```

2. Ends with Query

To find all Person entities where the name ends with a specific string (e.g., "Smith"), you can use:

```
@Query("SELECT p FROM Person p WHERE p.name LIKE %:nameSuffix")
```

```
List<Person> findByNameEndsWith(@Param("nameSuffix") String nameSuffix);
```

3. Using Spring Data JPA Method Naming

Alternatively, Spring Data JPA provides method naming conventions that you can use without writing a query explicitly:

- Starts with: `findByNameStartingWith(String namePrefix)`
- Ends with: `findByNameEndingWith(String nameSuffix)`

Example:

```
public interface PersonRepository extends JpaRepository<Person, Long> {  
    List<Person> findByNameStartingWith(String namePrefix);
```

```
    List<Person> findByNameEndingWith(String nameSuffix);  
}
```

```
public interface PersonRepository extends JpaRepository<Person, Long> {  
    List<Person> findByNameStartingWith(String namePrefix);
```

```
    List<Person> findByNameEndingWith(String nameSuffix);  
}
```

Usage Example:

```
@Autowired
```

```
private PersonRepository personRepository;
```

```
public void exampleUsage() {
```

```
    List<Person> startsWithJohn = personRepository.findByNameStartingWith("John");
```

```
    List<Person> endsWithSmith = personRepository.findByNameEndingWith("Smith");  
}
```

This setup allows you to perform queries based on the beginning or end of string values in the database fields.

JPQL with @Query Annotation

JPQL (Java Persistence Query Language) is a query language defined as part of the Java Persistence API (JPA) specification. It allows developers to write database queries using entity classes instead of SQL queries. When using JPQL with Spring Data JPA, the @Query annotation is particularly useful as it allows developers to define JPQL queries directly in repository interfaces or classes.

The @Query annotation in Java is used in conjunction with JPQL (Java Persistence Query Language) to define custom queries directly on repository methods within Spring Data JPA.

This powerful feature allows developers to write complex queries without needing to create separate query methods, providing a clean and efficient way to interact with the database.

Key points

- ✓ Custom Queries
- ✓ Flexibility
- ✓ Parameter Binding
- ✓ Readability

Example

Consider a scenario in a customer management system where you need to find all customers who have made purchases over a certain amount in the last year. Instead of writing multiple methods or complex query logic in the service layer, you can define a custom query using the @Query annotation in your repository.

In this example, the @Query annotation allows you to specify a JPQL query that filters customers based on purchase history and amount, ensuring that only relevant data is fetched from the database. This approach simplifies the repository interface while providing the flexibility to handle complex query requirements.

Why use JPL with @Query

Efficiency: Simplifies the process of writing and managing complex queries.

Conciseness: Reduces boilerplate code and keeps the repository interface clean.

Maintainability: Enhances code readability and maintainability by centralizing query definitions.

Using JPQL with the @Query annotation enables efficient and flexible data retrieval, making it an essential tool for developers working with complex database interactions in Java applications.

Advantages and Disadvantages

Advantages:

1. Type Safety:

JPQL queries are validated at compile-time, ensuring type safety. This means errors related to query syntax or incorrect entity mappings can be caught early in the development process.

2. Object-Oriented Approach:

JPQL queries are written using entity classes and their fields, which makes the queries more object-oriented compared to SQL queries. This aligns well with the Java programming language and object-relational mapping (ORM) principles.

3. Database Independence:

JPQL queries are database-independent, which means you can write the same query regardless of the underlying database vendor. The JPA provider (like Hibernate) translates JPQL queries to native SQL queries suitable for the database in use.

4. Integration with Spring Data JPA:

The @Query annotation integrates seamlessly with Spring Data JPA repositories, allowing you to define queries directly in repository interfaces or classes. This promotes clean and maintainable code by keeping data access logic centralized.

5. Named Parameters and Pagination Support:

JPQL queries support named parameters (?1, ?2, etc.) and pagination (LIMIT, OFFSET in SQL), making it easier to handle complex queries and optimize database access.

6. Reduced Boilerplate Code:

By using JPQL queries with Spring Data JPA, developers can reduce boilerplate code associated with traditional JDBC or raw SQL queries, such as result set mapping and resource management.

Disadvantages:

1. Learning Curve:

Developers new to JPA and JPQL may face a learning curve, especially understanding the nuances of object-relational mapping and JPQL syntax. This can lead to initial difficulties in writing and debugging queries.

2. Limited Native SQL Features:

JPQL is intentionally limited in features compared to native SQL queries. Some complex SQL operations or vendor-specific optimizations might be challenging or impossible to express in JPQL. In such cases, native queries (using `@Query(nativeQuery = true)`) or stored procedures may be required.

3. Potential Performance Implications:

Poorly optimized JPQL queries can lead to performance issues, especially when dealing with large datasets or complex relationships. Understanding how JPA translates JPQL to SQL and how indexes are used is crucial for optimizing query performance.

4. Maintenance of Queries:

While the `@Query` annotation centralizes query definitions, maintaining queries scattered across multiple repositories can become cumbersome in large applications. It's essential to balance between readability and maintainability.

5. Limited SQL-Specific Features:

JPQL lacks certain advanced SQL features such as window functions, specific optimizations (like hints), and some database-specific functions. When such features are necessary, native queries or stored procedures might be more suitable.

Steps to create an Application

The following basic setup provides a foundation for creating a Spring Boot application that uses Spring Data JPA with JPQL queries (`@Query` annotation). Customize the entity classes, repository methods, and service logic based on your application requirements. You can expand this setup by adding more entities, repositories, services, and controllers as needed.

1. Setup Your Development Environment:

- ✓ Ensure you have Java Development Kit (JDK) installed (preferably JDK 8 or later).
- ✓ Set up your IDE (like IntelliJ IDEA, Eclipse, or Spring Tools Suite).

2. Create a New Spring Boot Project:

- ✓ You can create a new Spring Boot project using Spring Initializr (<https://start.spring.io/>) or directly in your IDE.
- ✓ Select the following dependencies:
 - Spring Web
 - Spring Data JPA
 - H2 Database (or any other database driver you prefer)

3. Define Entity Classes:

- ✓ Define your entity classes with annotations (@Entity, @Id, @GeneratedValue, etc.). These classes represent your database tables.

4. Create a Repository Interface:

- ✓ Create a repository interface that extends JpaRepository (or a suitable repository interface).
- ✓ Define JPQL queries using the @Query annotation within methods in this interface.

5. Service Layer (Optional):

- ✓ Implement a service layer to encapsulate business logic.
- ✓ Use the repository methods (with JPQL queries) in the service layer to fetch or manipulate data.

6. Controller Layer (Optional):

- ✓ Create RESTful endpoints using Spring MVC (@RestController, @RequestMapping, etc.) if you want to expose your data via HTTP endpoints.

7. Configure Application Properties:

- ✓ Configure database connection details (spring.datasource.* properties) in application.properties or application.yml.

8. Run and Test Your Application:

- ✓ Run your Spring Boot application and test the JPQL queries to ensure they work as expected.
- ✓ Use tools like Postman or curl to test RESTful endpoints if you have created them.

Example

Implement a Spring Boot application for managing a library's book inventory for the following,

1. Entity Definition:

- ✓ Create an entity class Book with the following attributes:
 - id (Long, primary key, auto-generated)
 - title (String)
 - author (String)

2. Repository Interface:

- ✓ Define a repository interface BookRepository extending JpaRepository for Book entities.
- ✓ Implement the following JPQL queries using the @Query annotation:
 - List<Book> findByAuthor(String authorName): Fetch all books written by a specific author.
 - List<Book> findByTitle(String title): Fetch all books whose title contains a specified keyword.

3. Service Layer (Optional):

- ✓ Implement a service class BookService that interacts with BookRepository.
- ✓ Include methods:
 - List<Book> findBooksByAuthor(String authorName): Invoke findByAuthor from BookRepository.
 - List<Book> findBooksByTitle(String title): Invoke findByTitle from BookRepository.

4. Main Application Setup:

- ✓ Configure the main application class Application with @SpringBootApplication.
- ✓ Set up application.properties or application.yml for database connection details (H2 or any preferred database).

5. Testing and Output:

- ✓ Create a simple test scenario in the main method of Application to demonstrate functionality:
 - Instantiate a few Book objects and save them using BookRepository.
 - Call methods from BookService to retrieve books by author and title.
 - Print the results to verify the correct operation of JPQL queries.

Program

1. Define Entity Classes:

Assume we have an entity class named Book:

@Entity

public class Book {

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

private Long id;

private String title;

private String author;

// Getters and setters

}

2. Create a Repository Interface:

Create a repository interface that extends JpaRepository and use the @Query annotation to define JPQL queries:

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import java.util.List;

public interface BookRepository extends JpaRepository<Book, Long> {
    @Query("SELECT b FROM Book b WHERE b.author = ?1")
    List<Book> findByAuthor(String authorName);

    @Query("SELECT b FROM Book b WHERE b.title LIKE %?1%")
    List<Book> findByTitle(String title);
}
```

In the repository interface:

- ✓ findByAuthor(String authorName): This method uses JPQL to fetch all Book entities where the author's name matches authorName.
- ✓ findByTitle(String title): This method uses JPQL to fetch all Book entities where the title contains title.

3. Usage in Service or Controller:

You can now use these methods in your service or controller classes. Here's an example of how you might use them in a service class:

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class BookService {
    @Autowired
    private BookRepository bookRepository;

    public List<Book> findBooksByAuthor(String authorName) {
        return bookRepository.findByAuthor(authorName);
    }

    public List<Book> findBooksByTitle(String title) {
        return bookRepository.findByTitle(title);
    }
}
```


Example Output

Assume we have some sample data in our database:

Output:

Books by Craig Walls:

Spring in Action by Craig Walls

Books with 'Java' in the title:

Java Programming by John Smith

Custom JPQL Queries

Custom JPQL queries are manually written JPQL statements that allow you to perform more complex operations or retrieve more specific data than what is possible with standard repository methods. In JPA, you can use the `@Query` annotation to define custom JPQL queries directly in the repository interface.

Custom JPQL (Java Persistence Query Language) queries allow developers to perform complex database operations tailored to specific application requirements. JPQL is similar to SQL but operates on the entity model rather than directly on database tables.

Custom JPQL queries enable fine-grained control over data retrieval and manipulation, enhancing the flexibility and power of JPA.

Key points

- Entity-Centric
- Flexibility
- Portability
- Reusability

Example

Consider an e-commerce application that needs to generate a sales report. The report requires filtering orders based on various criteria such as date range, customer status, and order total. Using custom JPQL queries, the application can dynamically build these queries to fetch the required data.

For instance, the sales manager wants to see all orders placed by VIP customers in the last month with a total value exceeding \$500. A custom JPQL query can be crafted to join the Order and Customer entities, apply the necessary filters, and aggregate the results.

Why use Custom JPQL Queries

Precision: Tailors database interactions precisely to application needs, ensuring optimal data retrieval.

Complex Conditions: Capable of handling intricate conditions and multiple entity relationships.

Scalability: Supports building scalable applications by allowing fine-tuned queries and efficient data access patterns.

Custom JPQL queries are essential for developing sophisticated and responsive Java applications, enabling developers to harness the full potential of JPA for complex data operations.

Step-by-Step Example

Let's consider a scenario where you have a simple application that manages a library of books and their authors. We want to perform some specific queries on the Book and Author entities that cannot be achieved through standard JPA methods.

Step 1: Define the EntitiesFirst, define the Book and Author entities with a one-to-many relationship between them.

```
import javax.persistence.*;  
import java.math.BigDecimal;  
import java.util.List;
```

```

@Entity
public class Book
{
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    private String isbn;
    private BigDecimal price;
    @ManyToOne
    @JoinColumn(name = "author_id")
    private Author author;
    // Getters and setters
}

@Entity
public class Author
{
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    @OneToMany(mappedBy = "author")
    private List<Book> books;
    // Getters and setters
}

```

Step 2: Create the Repository Interfaces

Next, create repository interfaces for Book and Author. Here, we'll focus on custom JPQL queries for the Book repository.

```

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import java.math.BigDecimal;
import java.util.List;
public interface BookRepository extends JpaRepository<Book, Long>
{
    // Custom JPQL Query to find books by a keyword in the title
    @Query("SELECT b FROM Book b WHERE b.title LIKE %:keyword%") List<Book>
    findBooksByTitleContaining(@Param("keyword") String keyword);
    // Custom JPQL Query to find books cheaper than a specified price
    @Query("SELECT b FROM Book b WHERE b.price < :price")
    List<Book> findBooksByPriceLessThan(@Param("price") BigDecimal price);
}

```

```
// Custom JPQL Query to calculate the average price of books by a specific author
@Query("SELECT AVG(b.price) FROM Book b WHERE b.author.name = :authorName")
BigDecimal findAverageBookPriceByAuthor(@Param("authorName") String authorName);
}
```

Step 3: Using the Custom Queries

These queries can now be used in your service layer to retrieve and manipulate data based on the custom JPQL queries defined.

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.math.BigDecimal; import java.util.List;
@Service
public class BookService
{
    @Autowired
    private BookRepository bookRepository;
    public List<Book> searchBooksByTitleKeyword(String keyword)
    {
        return bookRepository.findBooksByTitleContaining(keyword);
    }
    public List<Book> findAffordableBooks(BigDecimal price)
    {
        return bookRepository.findBooksByPriceLessThan(price);
    }
    public BigDecimal getAveragePriceByAuthor(String authorName)
    {
        return bookRepository.findAverageBookPriceByAuthor(authorName);
    }
}
```

Step 4: Testing the Queries

You can test the queries by calling the service methods in your controller or directly from a test class.

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import java.math.BigDecimal;
import java.util.List;
@RestController
public class BookController
{
    @Autowired
    private BookService bookService;
    @GetMapping("/books/search")
    public List<Book> searchBooks(@RequestParam String keyword)
    {
```

```

        return bookService.searchBooksByTitleKeyword(keyword);
    }

    @GetMapping("/books/affordable")
    public List<Book> findAffordableBooks(@RequestParam BigDecimal price)
    {
        return bookService.findAffordableBooks(price);
    }
    @GetMapping("/books/average-price")
    public BigDecimal getAveragePriceByAuthor(@RequestParam String authorName)
    {
        return bookService.getAveragePriceByAuthor(authorName);
    }
}

```

Key Points

JPQL Syntax:

JPQL syntax is similar to SQL but operates on entity objects rather than databasetables. Use entity names and field names in the queries.

@Query Annotation:

The @Query annotation allows you to define custom queries directly in the repository interface.

Parameters:

Parameters are passed to JPQL queries using the @Param annotation.

Use Cases:

Custom JPQL queries are useful for complex searches, aggregations, and any query that is not covered by JPA's built-in query methods.

10. ASSIGNMENT – 1 To 5

Type 1: Implement Pagination & Sorting in a Spring Boot JPA Application

- **CO Level:** CO3 – Apply JPA features to develop efficient data access layers.
- **K Level:** K3 – Apply.

❑ Question:

Develop a Spring Boot application that retrieves customer records from a database using JPA with pagination and sorting. Implement both static sorting (predefined) and dynamic sorting (user-specified).

❑ Key Points:

1. Use Pageable and PageRequest classes.
2. Demonstrate ascending & descending sort orders.
3. Handle edge cases like empty pages or invalid page numbers.
4. Test with both large and small datasets.

Type 2: Demonstrate the Use of @Transient Annotation in JPA

- **CO Level:** CO2 – Demonstrate understanding of JPA annotations for entity mapping.
- **K Level:** K2 – Understand.

❑ Question:

Create a JPA entity Product that has a calculated field discountedPrice which should not be persisted in the database. Use the @Transient annotation to achieve this and explain its purpose in JPA.

❑ Key Points:

1. @Transient prevents persistence of fields.
2. Example with a calculated property.
3. Difference between @Transient and transient keyword in Java.
4. Where and why @Transient is useful.

Type 3: Write JPQL Queries using @Query Annotation

- **CO Level:** CO4 – Develop custom queries for complex data retrieval using JPQL.
- **K Level:** K4 – Analyze.
- ❑ **Question:**

Create a repository interface for the Employee entity with custom JPQL queries using @Query. Include: •

 - A query to fetch employees with salary greater than a given amount.
 - A query to fetch employees by department name.
 - A query to update employee email by ID.
- ❑ **Key Points:**
 1. Syntax of JPQL (entity-based, not table-based).
 2. Positional and named parameters (?1, :paramName).
 3. Differences between @Query and derived query methods.
 4. Handling @Modifying for update/delete queries.

Type 4: Implement a “Starts With” and “Ends With” Query in JPA

- **CO Level:** CO3 – Apply advanced query capabilities in JPA.
- **K Level:** K3 – Apply.
- ❑ **Question:**

Using Spring Data JPA, create repository methods to find all Customer records whose names start with a given prefix and end with a given suffix. Demonstrate both method naming convention and @Query annotation approaches.
- ❑ **Key Points:**
 1. Method naming: findByNameStartingWith(String prefix) and findByNameEndingWith(String suffix).
 2. Using LIKE with wildcards in JPQL: LIKE :prefix% and %:suffix.
 3. Performance considerations for LIKE queries.
 4. Case sensitivity handling.

10. ASSIGNMENT – 1 To 5

Type 5: Create a Custom JPQL Query with Multiple Joins

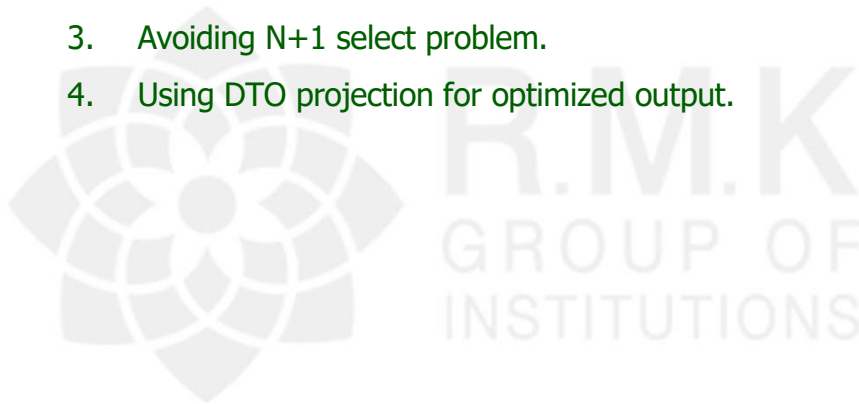
- **CO Level:** CO5 – Design and optimize complex JPA queries for relational data.
- **K Level:** K5 – Evaluate.

❑ Question:

Write a JPQL query to retrieve all orders along with customer details where the order total is above a given amount, and the customer belongs to a specific city. Implement this using the @Query annotation with multiple joins.

❑ Key Points:

1. JOIN and JOIN FETCH in JPQL.
2. Filtering results with WHERE clauses.
3. Avoiding N+1 select problem.
4. Using DTO projection for optimized output.



11. PART A : Questions & Answers

1. What is pagination in JPA?(CO2, K2)

The process of retrieving records in chunks (pages) instead of all at once.

2. Which Spring Data JPA interface supports pagination? (CO2, K1)

Pageable interface.

3. How do you sort results in JPA? (CO2, K3)

Use Sort class or PageRequest.of(page, size, Sort.by("field")).

4. Difference between Page and Slice? (CO2, K2)

Page contains total pages and count; Slice only has current data & next slice info.

5. Example method for pagination in JPA? (CO2, K3)

```
Page<User> findAll(Pageable pageable);
```

6. Purpose of @Transient in JPA? (CO3, K1)

Marks a field that is not persisted in DB.

7. Package name of @Transient? (CO3, K1)

javax.persistence.Transient (Jakarta in new versions)

8. Difference between @Transient and Java transient? (CO3, K2)

@Transient is JPA-specific; transient is for Java serialization.

9. Can @Transient fields be used in JPQL? (CO3, K2)

No, because they are not stored in DB.

10. Example use case for @Transient? (CO3, K3)

Storing calculated values like fullName without DB storage.

11. What is JPQL? (CO4, K1)

Java Persistence Query Language for querying entity objects.

12. Difference between JPQL and SQL? (CO4, K2)

JPQL uses entities/fields; SQL uses tables/columns.

13. Annotation to define JPQL in repository? (CO4, K1)

@Query annotation.

14. JPQL query to fetch all entities? (CO4, K3)

```
SELECT e FROM EntityName e.
```

15. How to pass parameters in JPQL?(CO4, K3)

Use @Param with named parameters (:name).

16. Method to find names starting with "A"?(CO5, K3)

findByNameStartingWith("A").

17. Method to find names ending with "z"? (CO5, K3)

findByNameEndingWith("z").

18. Method to find names containing "tech"? (CO5, K3)

findByNameContaining("tech").

19. Example of custom JPQL query with @Query? (CO6, K4)

@Query("SELECT u FROM User u WHERE u.age > :age")

20. Can JPQL queries use joins? Give example. (CO6, K4)

Yes, e.g., SELECT o FROM Order o JOIN o.customer c WHERE c.name = :name.

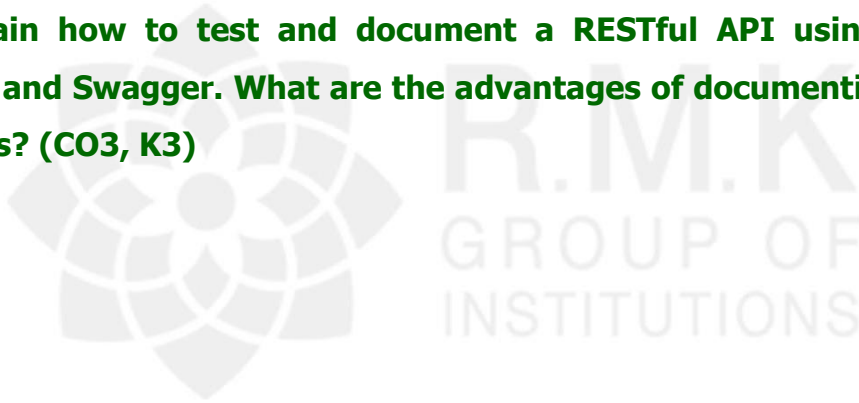


11. PART B QUESTIONS

1. Explain with an example how to implement pagination in a Spring Data JPA repository for fetching employee records. Include the method signature and controller code. (CO3, K3)
2. Demonstrate sorting records in ascending and descending order using Spring Data JPA's Sort class. (CO3, K3)
3. Write a Spring Boot method that retrieves student details in pages of size 5 and sorts them by lastName in ascending order. (CO3, K3)
4. Explain the purpose of the @Transient annotation in JPA. Provide a practical example of when and why it should be used in an entity class. (CO3, K4)
5. Compare the @Transient annotation with the Java transient keyword, highlighting differences in functionality and scope. (CO3, K3)
6. Implement a query method in a repository to find all customer names starting with a given prefix. (CO3, K3)
7. Implement a query method to fetch all product names that end with a specific string. (CO3, K4)
8. Write a custom JPQL query using the @Query annotation to fetch all users who have registered in the last 7 days. (CO3, K3)
9. Demonstrate how to use named parameters in a JPQL query with @Query annotation for fetching employees by department name. (CO3, K3)
10. Show an example of using positional parameters in JPQL with @Query to get orders by customer ID. (CO3, K3)

12. PART B QUESTIONS

11. Write a JPQL query that joins two entities (Order and Customer) to fetch orders along with customer names. (CO3, K3)
12. Differentiate between JPQL and native queries. Give an example of a native SQL query in a JPA repository method. (CO3, K4)
13. Write a method that accepts a column name and sort direction from the user and returns paginated, sorted results dynamically.? (CO3, K3)
14. Consider a database with 1 million product records. Explain how you would implement pagination, sorting, and filtering efficiently using JPA and Spring Boot. (CO3, K3)
15. Explain how to test and document a RESTful API using tools like Postman and Swagger. What are the advantages of documenting your API endpoints? (CO3, K3)



12. GATE TYPE - QUESTIONS

Q1. In Spring Data JPA, which of the following interfaces is used to represent a paginated result set? (CO3,K2)

- a) Pageable
- b) Page
- c) Slice
- d) List

Answer: b) Page

Q2. In JPA, to fetch the second page of size 10 sorted by name ascending, which code snippet is correct? (CO3,K2)

- a) `PageRequest.of(2, 10, Sort.by("name"))`
- b) `PageRequest.of(1, 10, Sort.by("name"))`
- c) `PageRequest.of(10, 2, Sort.by("name"))`
- d) `PageRequest.of(0, 10, Sort.by("name"))`

Answer: b) `PageRequest.of(1, 10, Sort.by("name"))`

Q3. The @Transient annotation in JPA is used to:

- a) Mark a field as non-nullable
- b) Indicate the field should not be persisted to the database
- c) Specify column type
- d) Map to a temporary table

Answer: b) Indicate the field should not be persisted to the database

Q4. In Spring Data JPA, which of the following methods can be used to perform pagination?

- a) `findAll(Pageable pageable)`
- b) `findAll(Sort sort)`
- c) `findAll(Page page, Sort sort)`
- d) `findById(Pageable pageable)`

Answer: a – `findAll(Pageable pageable)`

Q5. The @Transient annotation in JPA is used to:

- a) Mark a field as primary key
- b) Exclude a field from persistence
- c) Enable caching of an entity
- d) Mark a column as nullable

Answer: b – Exclude a field from persistence

Q6. Which JPQL query correctly finds employees whose names start with 'A'?

- a) `SELECT e FROM Employee e WHERE e.name LIKE 'A%'`
- b) `SELECT e FROM Employee e WHERE e.name STARTS 'A'`
- c) `SELECT e FROM Employee e WHERE e.name = 'A*'`
- d) `SELECT e FROM Employee e WHERE e.name STARTS WITH 'A'`

Answer: a – Uses `LIKE 'A%'` for "starts with" functionality.

Q7. Which JPQL statement is valid?

- a) `SELECT * FROM Employee`
- b) `SELECT e FROM Employee e`
- c) `SELECT e.name, e.salary FROM Employee e`
- d) Both b and c

Answer: d – JPQL uses entity names (not `*`) and allows selecting specific fields.

Q8. Which of the following PageRequest creations will fetch the first 20 records sorted by createdAt in descending order?

- a) `PageRequest.of(1, 20, Sort.by("createdAt").descending())`
- b) `PageRequest.of(0, 20, Sort.by("createdAt").descending())`
- c) `PageRequest.of(0, 10, Sort.by("createdAt").descending())`
- d) `PageRequest.of(1, 10, Sort.by("createdAt").descending())`

Answer: b – `PageRequest.of(0, 20, ...)` fetches page index 0 (first page) with 20 records.

Q9. Which JPQL query is valid for finding customers whose names contain "tech"?

- a) `SELECT c FROM Customer c WHERE c.name LIKE '%tech%'`
- b) `SELECT c FROM Customer c WHERE c.name CONTAINS 'tech'`
- c) `SELECT c FROM Customer c WHERE c.name = '*tech*'`
- d) `SELECT c FROM Customer c WHERE c.name LIKE 'tech'`

Answer: a – JPQL uses SQL `LIKE` syntax with `%` for partial matches.

Q10. Which of the following repository methods is correct for fetching all products sorted by price descending?

- a) `List<Product> findAllOrderByPriceDesc();`
- b) `List<Product> findByPriceDesc();`
- c) `List<Product> findAllSortedByPriceDesc();`
- d) `List<Product> sortByPriceDesc();`

Answer: a – Spring Data JPA method naming supports `findAllOrderBy<Field>Desc()`.

13. SUPPORTIVE ONLINE CERTIFICATION COURSES

Sl. No.	Courses	Platform
1	Java Spring Boot	Infosys Springboard
2	Spring MVC, Spring Boot and Rest Controllers	Coursera
3	Master Java Web Services and REST API with Spring Boot	Udemy



R.M.K.
GROUP OF
INSTITUTIONS

15. REAL TIME APPLICATIONS

- ❖ RESTful APIs are commonly used in web and mobile applications to retrieve or modify resources and data on remote systems. Some examples include:
- ❖ Social media sites like Twitter, Facebook use REST APIs to integrate with third-party applications and allow posting updates.
- ❖ Ridesharing apps like Uber and Lyft use REST APIs to coordinate cars, obtain maps, fares and location data.
- ❖ Video, music streaming through Netflix, Spotify use REST APIs to get info on media files from servers.
- ❖ Banking apps utilize REST APIs to retrieve your account data and initiate transactions with remote servers.
- ❖ IoT apps interface with sensors and devices via REST APIs to monitor them and send commands.

16. CONTENT BEYOND SYLLABUS

RESTful API Versioning (URI versioning, header versioning)

Versioning is a strategy used in RESTful APIs to maintain backward compatibility when changes are made to the API, such as updating features, modifying response formats, or changing data structures.

Why Versioning Is Needed:

- To avoid breaking existing clients using the old API.
- To support new features while keeping older versions running.
- To test or roll out features incrementally.

Common Versioning Techniques:

1. URI Versioning (Path Versioning)

Version info is included in the URL path.

Example:

GET /api/v1/students

GET /api/v2/students

2. Header Versioning

Version is passed in a custom HTTP header.

Example:

GET /api/students

Header: X-API-VERSION:

Pros:

1. Keeps the URI clean and consistent.
2. Aligns with HTTP protocol standards.

Cons:

1. Harder to test directly in browser.
2. Requires clients to set headers properly.

16. CONTENT BEYOND SYLLABUS

3. Media Type Versioning (a.k.a. Content Negotiation)

Version is passed using the Accept header in the format of custom MIME types.

Example:

GET /api/students

Accept: application/vnd.school.v1+json

Pros:

1. RESTful and elegant.
2. Keeps URI clean.

Cons:

1. Complex setup.
2. Tooling and documentation may be harder to manage.

❖ Versioning in Spring Boot Example

❖ URI Versioning:

@RestController

```
public class StudentControllerV1 {  
    @GetMapping("/api/v1/student")  
    public StudentV1 getStudentV1() {  
        return new StudentV1("John Doe");  
    }  
}
```

}

@RestController

```
public class StudentControllerV2 {  
    @GetMapping("/api/v2/student")  
    public StudentV2 getStudentV2() {  
        return new StudentV2(new Name("John", "Doe"));  
    }  
}
```

}

16. CONTENT BEYOND SYLLABUS

❖ Header Versioning:

@RestController

```
public class StudentHeaderController {
```

```
    @GetMapping(value = "/api/student", headers = "X-API-VERSION=1")
```

```
    public StudentV1 getStudentV1() {
```

```
        return new StudentV1("John Doe");
```

```
    }
```

```
    @GetMapping(value = "/api/student", headers = "X-API-VERSION=2")
```

```
    public StudentV2 getStudentV2() {
```

```
        return new StudentV2(new Name("John", "Doe"));
```

```
    }
```

```
}
```



17. ASSESSMENT SCHEDULE

- **Tentative schedule for the Assessment During
2025-2026 Odd Semester**

S.NO	Name of the Assessment	Start Date	End Date	Portion
1	Unit Test 1	As per schedule after Unit I completion		UNIT 1
2	IAT 1	As per Academic Calendar		UNIT 1 & 2
3	Unit Test 2	As per schedule after Unit III completion		UNIT 3
4	IAT 2	As per Academic Calendar		UNIT 3 & 4
5	Model	As per Academic Calendar		ALL 5 UNITS

18. PRESCRIBED TEXT BOOKS & REFERENCE BOOKS

TEXT BOOKS:

1. Raja CSP Raman, Ludovic Dewailly, "Building RESTful Web Services with Spring 5", Packt Publishing, 2018.
2. Leonard Richardson, Sam Ruby "RESTful Web Services" O'Reilly Media, 2008.
3. Ludovic Dewailly, "Building a RESTful Web Service with Spring: A hands-on guide to building an enterprise-grade, scalable RESTful web service using the Spring Framework", Packt Publishing, 2015
4. Raja CSP Raman, Ludovic Dewailly, "Building RESTful Web Services with Spring 5 - Second Edition
5. Leverage the power of Spring 5.0, Java SE 9, and Spring Boot 2.0", Packt Publishing, 2018.

REFERENCES:

1. Ranga Karanam, "Master Java Web Services and REST API with Spring Boot", Packt Publishing, 2018.
2. Balaji Varanasi, Sudha Belida, "Spring REST", Apress, 2015.
3. Greg L. Turnquist, "Learning Spring Boot 2.0", Packt Publishing, 2021.
4. Sourabh Sharma, "Modern API Development with Spring and Spring Boot", Packt Publishing, 2021.

19. MINI PROJECT – 1 to 5

Title 1 : "Online Bookstore API with Pagination and Sorting" (CO3,K4)

Description:

Develop a REST API for managing books in an online bookstore. Implement **pagination and sorting** to list books efficiently (e.g., sort by price, title, or publication date). Use **JPQL @Query** for custom searches like "find books starting with a specific letter" or "ending with a certain word." Include @Transient fields for calculated attributes like discount price without storing them in the DB.

Title 2 : Student Records Management API (CO3,K3)

Description:

Build a REST API to manage student data for a university. Implement pagination and sorting for browsing student lists (by name, GPA, or admission year). Use starts-with and ends-with JPA queries to filter students by first or last name. Include @Transient fields for dynamically calculated fields like "current semester" based on admission date.

Title 3: Employee Directory with Advanced Search (CO3,K3)

Description:

Create an employee directory REST API for a company. Support **pagination and sorting** of employee lists (by department, join date, salary). Use **custom JPQL queries** for advanced searches, such as finding employees whose names start with 'A' or end with 'son'. Implement @Transient attributes for non-persistent computed data like "years of service."

19. MINI PROJECT – 1 to 5

Title 4: E-commerce Order Tracking API (CO3,K3)

Description:

Design a REST API for managing customer orders in an e-commerce platform. Implement **pagination and sorting** to view orders by date or total amount. Use **JPA queries** with starts-with and ends-with to search orders by customer name or product code. Include @Transient for runtime-only fields like "expected delivery date" calculated from order date.

Title 5: Library Member Management API (CO3,K3)

Description:

Develop a REST API to handle library membership records. Provide **paginated and sorted** member lists (by join date, name, or membership type). Implement **custom JPQL queries** for finding members whose names match certain patterns. Use @Transient for fields like "membership validity days left," calculated dynamically without persisting.



Thank you

Disclaimer:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.